Full-Stack Hackathon Cheatsheet

Stack: Next.js + Auth.js + Prisma + PostgreSQL (Supabase) + Tailwind CSS + shadcn/ui + Zod + Chart.js + Vercel

> *One file. Every command. Copy-paste and go. Built around a **Todo App with Role-Based Access Control (RBAC) + Analytics Dashboard** as the running example.*

Table of Contents

1. Why This Stack?
2. Project Setup from Scratch
3. Git + GitHub Setup
4. Install All Dependencies
5. Supabase + PostgreSQL Setup
6. Prisma Setup + Schema
7. Auth.js (NextAuth v5) Setup
8. shadcn/ui Setup
9. Zod Validation Schemas
10. Database Layer (Prisma Client)
11. Folder Structure
12. RBAC Implementation
13. Todo CRUD with Server Actions
14. Building the Pages
15. Chart.js Analytics Dashboard
16. Tailwind Patterns
17. Vercel Deployment
18. Quick Reference Command Table
19. Common Errors + Fixes
20. Hackathon Checklist

---

1. Why This Stack?

Tool	What It Does	Why You'd Pick It
Next.js 14+	React framework with server-side rendering, routing, server actions	Zero-config React. File-based routing = fast prototyping. Server actions = no separate API needed.
Auth.js (NextAuth v5)	Authentication library	Handles Google/GitHub/Email login in ~30 lines. Session management built-in. Works with App Router.
Prisma	Database ORM (Object-Relational Mapper)	Write TypeScript instead of raw SQL. Auto-generates types. Migration system keeps DB in sync.
PostgreSQL	Relational database	Industry standard. Handles complex queries. Free tier on Supabase.
Supabase	Hosted PostgreSQL + extras	Free PostgreSQL database in 30 seconds. No server management. Connection string ready to go.
Tailwind CSS	Utility-first CSS framework	Style directly in JSX. No switching between files. Consistent design system.
shadcn/ui	Pre-built UI components	Copy-paste beautiful components (buttons, dialogs, tables). You own the code. Built on Tailwind.
Zod	Schema validation	Validate form data on client AND server with the same schema. TypeScript types auto-generated from schemas.
Chart.js	Charting library	Simple API for bar/pie/line charts. `react-chartjs-2` wrapper makes it React-friendly.
Vercel	Deployment platform	`git push` = deployed. Made by Next.js creators. Free tier handles hackathon traffic.

---

2. Project Setup from Scratch

Create Next.js App

```
npx create-next-app@latest my-todo-app
```

When prompted, select these options:

```
✔ Would you like to use TypeScript? → Yes
✔ Would you like to use ESLint? → Yes
✔ Would you like to use Tailwind CSS? → Yes
✔ Would you like your code inside a `src/` directory? → Yes
✔ Would you like to use App Router? → Yes
✔ Would you like to use Turbopack for next dev? → Yes
✔ Would you like to customize the import alias? → No (keep @/*)
```

Why these choices:

- **TypeScript** → Catches bugs before runtime. Prisma generates types for your DB models.
- **ESLint** → Catches code quality issues automatically.
- **Tailwind** → Already included, no separate setup.
- **src/ directory** → Keeps root clean. Config files stay outside, app code stays inside `src/`.
- **App Router** → Next.js 14+ standard. Supports server components, server actions, layouts.
- **Turbopack** → Faster dev server. Hot reload in milliseconds.

Enter the project

```
cd my-todo-app
```

Verify it works

```
npm run dev
```

Open `http://localhost:3000` — you should see the Next.js welcome page. Press `Ctrl+C` to stop.

---

3. Git + GitHub Setup

Initialize Git (already done by create-next-app, but just in case)

```
git init
```

Create `.gitignore` (create-next-app generates one, but make sure it has these)

```
# dependencies
node_modules/

# next.js
.next/
out/

# environment variables — NEVER commit these
.env
.env.local
.env.production

# prisma
prisma/*.db

# misc
.DS_Store
*.tsbuildinfo
```

Create a GitHub Repository

Option A: Using GitHub CLI (faster)

```
# Install GitHub CLI if you don't have it
# Windows: winget install GitHub.cli
# Mac: brew install gh

# Login
gh auth login

# Create repo and push (from your project folder)
gh repo create my-todo-app --public --source=. --remote=origin --push
```

Option B: Manual

1. Go to https://github.com/new (https://github.com/new)
2. Name it `my-todo-app`
3. Do NOT add README/gitignore (you already have them)
4. Click "Create repository"
5. Run these commands:

```
git add .
git commit -m "init: next.js project setup"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/my-todo-app.git
git push -u origin main
```

Why Git early? Every hackathon horror story involves losing code. Commit after every working feature.

Git Workflow During Hackathon

```
# After completing each feature:
git add .
git commit -m "feat: add user authentication"
git push

# Quick shortcut (stage + commit + push):
git add . && git commit -m "feat: add todo CRUD" && git push
```

Commit message prefixes:

- `feat:` — new feature
- `fix:` — bug fix
- `setup:` — configuration/setup changes
- `style:` — UI/styling changes

---

4. Install All Dependencies

Run this entire block at once:

```
# Auth
npm install next-auth@beta @auth/prisma-adapter

# Database
npm install @prisma/client
npm install prisma --save-dev

# Validation
npm install zod

# Charts
npm install react-chartjs-2 chart.js

# Password hashing (for email/password auth)
npm install bcryptjs
npm install @types/bcryptjs --save-dev
```

What each package does:

- `next-auth@beta` → Auth.js v5 (latest, works with App Router)
- `@auth/prisma-adapter` → Connects Auth.js to your Prisma database (stores users/sessions in PostgreSQL)
- `@prisma/client` → Auto-generated client to query your database from code
- `prisma` → CLI tool for migrations, schema management, Prisma Studio (dev-only)
- `zod` → Runtime type validation for forms and API inputs
- `react-chartjs-2` + `chart.js` → Charts (react-chartjs-2 is the React wrapper)
- `bcryptjs` → Hashes passwords before storing them (never store plain text passwords)

---

5. Supabase + PostgreSQL Setup

Create Supabase Project

1. Go to https://supabase.com (https://supabase.com) → Sign up (use GitHub login)
2. Click **"New Project"**
3. Fill in:
   - **Name**: `my-todo-app`
   - **Database Password**: Generate one and **SAVE IT SOMEWHERE**
   - **Region**: Pick the closest one to you
4. Click **"Create new project"** → Wait ~2 minutes

Get Connection Strings

1. In Supabase dashboard → Click **"Connect"** (top right green button)
2. Go to **"ORMs"** tab
3. Select **"Prisma"**
4. You'll see two URLs. Copy them.

Create `.env` file in your project root

```
# .env

# From Supabase → Connect → ORMs → Prisma
# Transaction mode (for Prisma queries)
DATABASE_URL="postgresql://postgres.[YOUR-PROJECT-REF]:[YOUR-PASSWORD]@aws-0-[REGION].pooler.supabase.com:6543/postgres?pgbouncer=tr

# Session mode (for Prisma migrations)
DIRECT_URL="postgresql://postgres.[YOUR-PROJECT-REF]:[YOUR-PASSWORD]@aws-0-[REGION].pooler.supabase.com:5432/postgres"

# Auth.js
AUTH_SECRET="run-this-command-to-generate: npx auth secret"
AUTH_URL="http://localhost:3000"

# Google OAuth (optional — get from console.cloud.google.com)
AUTH_GOOGLE_ID=""
AUTH_GOOGLE_SECRET=""

# GitHub OAuth (optional — get from github.com/settings/developers)
AUTH_GITHUB_ID=""
AUTH_GITHUB_SECRET=""
```

Generate AUTH_SECRET

```
npx auth secret
```

This adds `AUTH_SECRET` to your `.env` file automatically.

Why two database URLs?

- `DATABASE_URL` → Uses connection pooling (port 6543). Handles many simultaneous connections efficiently. Used for normal app queries.
- `DIRECT_URL` → Direct connection (port 5432). Required for schema migrations because Prisma needs a persistent connection to alter tables.

---

6. Prisma Setup + Schema

Initialize Prisma

```
npx prisma init
```

This creates:

- `prisma/schema.prisma` → Your database schema file
- Updates `.env` with a placeholder `DATABASE_URL` (you already set the real one)

Write the Schema

Replace everything in `prisma/schema.prisma` with:

```
// prisma/schema.prisma

// This tells Prisma which database to connect to and how to generate the client

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider  = "postgresql"
  url       = env("DATABASE_URL")      // Connection pooling URL
  directUrl = env("DIRECT_URL")        // Direct connection for migrations
}

// ─── ENUMS ──────────────────────────────────────

// Role enum — used for RBAC (Role-Based Access Control)
// ADMIN can manage all users and todos
// USER can only manage their own todos
enum Role {
  ADMIN
  USER
}

// Priority levels for todos
enum Priority {
  LOW
  MEDIUM
  HIGH
}

// ─── AUTH.JS REQUIRED TABLES ────────────────────

// These tables are required by Auth.js to store users, sessions, and OAuth accounts.
// Don't modify the field names — Auth.js expects these exact names.

model User {
  id            String    @id @default(cuid())
  name          String?
  email         String    @unique
  emailVerified DateTime?
  image         String?
  password      String?                        // Only used for email/password auth (null for OAuth users)
  role          Role      @default(USER)        // RBAC: every new user starts as USER

  // Relations
  accounts      Account[]
  sessions      Session[]
  todos         Todo[]

  createdAt     DateTime  @default(now())
  updatedAt     DateTime  @updatedAt
}

model Account {
  id                String  @id @default(cuid())
  userId            String
  type              String
  provider          String
  providerAccountId String
  refresh_token     String?
  access_token      String?
  expires_at        Int?
  token_type        String?
  scope             String?
```

```
  id_token          String?
  session_state     String?

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@unique([provider, providerAccountId])
}

model Session {
  id           String   @id @default(cuid())
  sessionToken String   @unique
  userId       String
  expires      DateTime

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
}

model VerificationToken {
  identifier String
  token      String
  expires    DateTime

  @@unique([identifier, token])
}

// ─── APP TABLES ──────────────────────────────────

// The actual Todo model for our app
model Todo {
  id          String   @id @default(cuid())
  title       String
  description String?
  completed   Boolean  @default(false)
  priority    Priority @default(MEDIUM)

  // Each todo belongs to a user
  userId      String
  user        User     @relation(fields: [userId], references: [id], onDelete: Cascade)

  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}
```

Schema explained:

- `@id @default(cuid())` → Auto-generates a unique ID for each record
- `@unique` → No two records can have the same value (e.g., email)
- `@default(now())` → Auto-sets to current timestamp when created
- `@updatedAt` → Auto-updates timestamp on every edit
- `@relation` → Links tables together (userId in Todo points to User.id)
- `onDelete: Cascade` → When a user is deleted, delete all their todos/sessions/accounts too

Push Schema to Database

```
# Create migration and apply it
npx prisma migrate dev --name init
```

What this does:

1. Compares your schema to the database
2. Generates SQL migration files (stored in `prisma/migrations/`)
3. Runs the SQL against your Supabase database
4. Generates the Prisma Client (TypeScript types for your models)

Useful Prisma Commands

```
# Open visual database browser (like phpMyAdmin but better)
npx prisma studio

# Regenerate client after schema changes (without migration)
npx prisma generate

# Reset database (DELETE ALL DATA + re-run migrations)
npx prisma migrate reset

# Push schema directly (skip migration file — good for prototyping)
npx prisma db push

# See migration status
npx prisma migrate status

# Format schema file
npx prisma format
```

When to use what:

- `migrate dev` → Normal development. Creates migration files so your team can sync.
- `db push` → Quick prototyping. Changes DB but no migration file. Use when you're iterating fast.
- `migrate reset` → Nuclear option. Wipes everything. Use when your DB is messed up.
- `studio` → Visual way to inspect/edit data. Great for debugging.

---

7. Auth.js (NextAuth v5) Setup

File: `src/lib/auth.ts`

This is the main Auth.js configuration. It sets up how users log in and how sessions work.

```
// src/lib/auth.ts

import NextAuth from "next-auth";
import { PrismaAdapter } from "@auth/prisma-adapter";
import Credentials from "next-auth/providers/credentials";
import Google from "next-auth/providers/google";
import GitHub from "next-auth/providers/github";
import bcrypt from "bcryptjs";
import { db } from "@/lib/db";

export const { handlers, signIn, signOut, auth } = NextAuth({
  // Prisma adapter stores users/sessions in your PostgreSQL database
  adapter: PrismaAdapter(db),

  // Use JWT strategy (not database sessions) — works better with Credentials provider
  session: { strategy: "jwt" },

  // Custom pages (instead of the default ugly Auth.js pages)
  pages: {
    signIn: "/login",
  },

  providers: [
    // ─── Google OAuth ───
    // Auto-reads AUTH_GOOGLE_ID and AUTH_GOOGLE_SECRET from .env
    Google,

    // ─── GitHub OAuth ───
    // Auto-reads AUTH_GITHUB_ID and AUTH_GITHUB_SECRET from .env
    GitHub,

    // ─── Email/Password ───
    Credentials({
      credentials: {
        email: {},
        password: {},
      },
      async authorize(credentials) {
        if (!credentials?.email || !credentials?.password) return null;

        // Find user by email
        const user = await db.user.findUnique({
          where: { email: credentials.email as string },
        });

        // User not found or no password (OAuth user trying to use password login)
        if (!user || !user.password) return null;

        // Compare password with hashed version in DB
        const isValid = await bcrypt.compare(
          credentials.password as string,
          user.password
        );

        if (!isValid) return null;

        // Return user object — this becomes the session user
        return user;
      },
    }),
  ],

  callbacks: {
    // JWT callback — runs when JWT is created or updated
    // We add the user's role and id to the token so it's available in the session
```

```
    async jwt({ token, user }) {
      if (user) {
        const dbUser = await db.user.findUnique({
          where: { email: token.email! },
        });
        if (dbUser) {
          token.role = dbUser.role;
          token.id = dbUser.id;
        }
      }
      return token;
    },

    // Session callback — runs when session is checked
    // We pass role and id from the token to the session object
    async session({ session, token }) {
      if (session.user) {
        session.user.role = token.role as string;
        session.user.id = token.id as string;
      }
      return session;
    },
  },
});
```

File: `src/types/next-auth.d.ts` — Extend Session Types

Auth.js doesn't know about your custom `role` field by default. This tells TypeScript about it.

```
// src/types/next-auth.d.ts

import { DefaultSession } from "next-auth";

declare module "next-auth" {
  interface Session {
    user: {
      id: string;
      role: string;
    } & DefaultSession["user"];
  }
}

declare module "next-auth/jwt" {
  interface JWT {
    role?: string;
    id?: string;
  }
}
```

File: `src/app/api/auth/[...nextauth]/route.ts` — API Route Handler

Auth.js needs an API route to handle OAuth callbacks, sign-in, sign-out, etc.

```
// src/app/api/auth/[...nextauth]/route.ts

import { handlers } from "@/lib/auth";

export const { GET, POST } = handlers;
```

File: `src/middleware.ts` — Protect Routes

This runs BEFORE every request. Redirects non-logged-in users away from protected pages.

```
// src/middleware.ts (in the root of src/, NOT inside app/)

import { auth } from "@/lib/auth";
import { NextResponse } from "next/server";

export default auth((req) => {
  const isLoggedIn = !!req.auth;
  const isOnDashboard = req.nextUrl.pathname.startsWith("/dashboard");
  const isOnAdmin = req.nextUrl.pathname.startsWith("/admin");
  const isOnAuth =
    req.nextUrl.pathname.startsWith("/login") ||
    req.nextUrl.pathname.startsWith("/register");

  // Redirect logged-in users away from login/register pages
  if (isOnAuth && isLoggedIn) {
    return NextResponse.redirect(new URL("/dashboard", req.nextUrl));
  }

  // Redirect non-logged-in users to login
  if (isOnDashboard && !isLoggedIn) {
    return NextResponse.redirect(new URL("/login", req.nextUrl));
  }

  // Redirect non-admin users away from admin pages
  if (isOnAdmin) {
    if (!isLoggedIn) {
      return NextResponse.redirect(new URL("/login", req.nextUrl));
    }
    if (req.auth?.user?.role !== "ADMIN") {
      return NextResponse.redirect(new URL("/dashboard", req.nextUrl));
    }
  }

  return NextResponse.next();
});

// Only run middleware on these routes (skip static files, images, etc.)
export const config = {
  matcher: ["/dashboard/:path*", "/admin/:path*", "/login", "/register"],
};
```

Register Action — Create New Users

```
// src/actions/auth.ts

"use server";

import bcrypt from "bcryptjs";
import { db } from "@/lib/db";
import { signIn } from "@/lib/auth";
import { AuthError } from "next-auth";
import { registerSchema } from "@/lib/validations";

export async function registerUser(formData: FormData) {
  const rawData = {
    name: formData.get("name") as string,
    email: formData.get("email") as string,
    password: formData.get("password") as string,
  };

  // Validate with Zod
  const validated = registerSchema.safeParse(rawData);
  if (!validated.success) {
    return { error: validated.error.errors[0].message };
  }

  const { name, email, password } = validated.data;

  // Check if email already exists
  const existingUser = await db.user.findUnique({ where: { email } });
  if (existingUser) {
    return { error: "Email already registered" };
  }

  // Hash password — 10 salt rounds
  const hashedPassword = await bcrypt.hash(password, 10);

  // Create user in database
  await db.user.create({
    data: { name, email, password: hashedPassword, role: "USER" },
  });

  // Auto-login after registration
  await signIn("credentials", { email, password, redirectTo: "/dashboard" });
}

export async function loginUser(formData: FormData) {
  const email = formData.get("email") as string;
  const password = formData.get("password") as string;

  try {
    await signIn("credentials", {
      email,
      password,
      redirectTo: "/dashboard",
    });
  } catch (error) {
    if (error instanceof AuthError) {
      return { error: "Invalid email or password" };
    }
    throw error; // Re-throws NEXT_REDIRECT (expected behavior)
  }
}
```

---

8. shadcn/ui Setup

Initialize shadcn

```
npx shadcn@latest init
```

When prompted:

```
✔ Which style would you like to use? → New York
✔ Which color would you like to use as base color? → Neutral
✔ Would you like to use CSS variables for colors? → Yes
```

Add Components You'll Need

```
# Add all at once
npx shadcn@latest add button card input label dialog table badge select textarea tabs separator avatar dropdown-menu sheet sonner
```

What each component is for:

- `button` → Click actions
- `card` → Content containers (todo cards, stat cards)
- `input` + `label` + `textarea` → Form fields
- `dialog` → Modal popups (create/edit todo)
- `table` → Admin data tables
- `badge` → Status tags (priority, role)
- `select` → Dropdowns (priority picker)
- `tabs` → Switch between views
- `separator` → Visual dividers
- `avatar` + `dropdown-menu` → User profile menu
- `sheet` → Slide-out mobile sidebar
- `sonner` → Success/error toast notifications (better API than Radix Toast)

Using a shadcn Component

```
// Example: Using the Button component
import { Button } from "@/components/ui/button";

export function MyComponent() {
  return (
    <div>
      <Button>Default</Button>
      <Button variant="destructive">Delete</Button>
      <Button variant="outline">Cancel</Button>
      <Button variant="ghost">Ghost</Button>
      <Button size="sm">Small</Button>
      <Button size="lg">Large</Button>
      <Button disabled>Disabled</Button>
    </div>
  );
}
```

---

9. Zod Validation Schemas

Zod lets you validate data on both client and server with the same schema. If validation fails, you get specific error messages.

```
// src/lib/validations.ts

import { z } from "zod";

// ─── Auth Schemas ──────────────────────────────

export const registerSchema = z.object({
  name: z
    .string()
    .min(2, "Name must be at least 2 characters")
    .max(50, "Name too long"),
  email: z.string().email("Invalid email address"),
  password: z
    .string()
    .min(6, "Password must be at least 6 characters")
    .max(100, "Password too long"),
});

export const loginSchema = z.object({
  email: z.string().email("Invalid email address"),
  password: z.string().min(1, "Password is required"),
});

// ─── Todo Schemas ──────────────────────────────

export const createTodoSchema = z.object({
  title: z
    .string()
    .min(1, "Title is required")
    .max(200, "Title too long"),
  description: z.string().max(1000, "Description too long").optional(),
  priority: z.enum(["LOW", "MEDIUM", "HIGH"]).default("MEDIUM"),
});

export const updateTodoSchema = z.object({
  id: z.string(),
  title: z.string().min(1).max(200).optional(),
  description: z.string().max(1000).optional(),
  completed: z.boolean().optional(),
  priority: z.enum(["LOW", "MEDIUM", "HIGH"]).optional(),
});

// ─── Generate TypeScript Types from Schemas ────────

// Instead of writing types manually, derive them from Zod schemas
// This guarantees your types always match your validation rules
export type RegisterInput = z.infer<typeof registerSchema>;
export type LoginInput = z.infer<typeof loginSchema>;
export type CreateTodoInput = z.infer<typeof createTodoSchema>;
export type UpdateTodoInput = z.infer<typeof updateTodoSchema>;
```

Why Zod?

- TypeScript types only exist at compile time. They're erased at runtime. A user can send `{ title: 123 }` and TypeScript won't catch it.
- Zod validates at RUNTIME. It actually checks the data when your server receives it.
- `z.infer<typeof schema>` auto-generates TypeScript types from your Zod schema, so you don't write the same types twice.

---

10. Database Layer (Prisma Client)

Prisma Client Singleton

```
// src/lib/db.ts

import { PrismaClient } from "@prisma/client";

// Problem: In development, Next.js hot-reloads your code. Each reload creates a NEW PrismaClient.
// After a few reloads, you run out of database connections.
// Solution: Store the client on `globalThis` so it persists across hot reloads.

const globalForPrisma = globalThis as unknown as { prisma: PrismaClient };

export const db = globalForPrisma.prisma || new PrismaClient();

if (process.env.NODE_ENV !== "production") {
  globalForPrisma.prisma = db;
}
```

Common Prisma Queries (Reference)

```
import { db } from "@/lib/db";

// ─── CREATE ───
const user = await db.user.create({
  data: { name: "John", email: "john@test.com", password: hashedPw },
});

const todo = await db.todo.create({
  data: { title: "Buy milk", userId: user.id },
});

// ─── READ ───
// Find one by unique field
const user = await db.user.findUnique({ where: { email: "john@test.com" } });

// Find one (first match)
const todo = await db.todo.findFirst({ where: { userId: "123", completed: false } });

// Find many with filtering, sorting, pagination
const todos = await db.todo.findMany({
  where: { userId: "123", completed: false },
  orderBy: { createdAt: "desc" },
  take: 10,       // limit
  skip: 0,        // offset
  include: { user: true },  // join user data
});

// Count
const count = await db.todo.count({ where: { userId: "123" } });

// ─── UPDATE ───
const updated = await db.todo.update({
  where: { id: "todo-id" },
  data: { completed: true },
});

// ─── DELETE ───
await db.todo.delete({ where: { id: "todo-id" } });

// Delete many
await db.todo.deleteMany({ where: { userId: "123", completed: true } });

// ─── AGGREGATE ───
const stats = await db.todo.groupBy({
  by: ["completed"],
  _count: { id: true },
  where: { userId: "123" },
});
// Result: [{ completed: true, _count: { id: 5 } }, { completed: false, _count: { id: 3 } }]
```

---

11. Folder Structure

This is the complete folder structure for the todo app. Create these folders and files:

```
my-todo-app/
├── prisma/
│   └── schema.prisma              # Database schema
├── src/
│   ├── actions/
│   │   ├── auth.ts                # Register + login server actions
│   │   └── todo.ts                # Todo CRUD server actions
│   ├── app/
│   │   ├── layout.tsx             # Root layout (wraps everything)
│   │   ├── page.tsx               # Landing page (/)
│   │   ├── login/
│   │   │   └── page.tsx           # Login page
│   │   ├── register/
│   │   │   └── page.tsx           # Register page
│   │   ├── dashboard/
│   │   │   ├── layout.tsx         # Dashboard layout (sidebar + header)
│   │   │   └── page.tsx           # User's todo list + mini chart
│   │   ├── admin/
│   │   │   ├── layout.tsx         # Admin layout
│   │   │   └── page.tsx           # Admin dashboard (all users, analytics)
│   │   └── api/
│   │       └── auth/
│   │           └── [...nextauth]/
│   │               └── route.ts   # Auth.js API handler
│   ├── components/
│   │   ├── ui/                    # shadcn components (auto-generated)
│   │   ├── navbar.tsx             # Top navigation bar
│   │   ├── sidebar.tsx            # Dashboard sidebar
│   │   ├── todo-card.tsx          # Single todo card component
│   │   ├── todo-form.tsx          # Create/edit todo form
│   │   ├── todo-list.tsx          # List of todo cards
│   │   ├── analytics-chart.tsx    # Chart.js charts
│   │   └── user-button.tsx        # User avatar + dropdown
│   ├── lib/
│   │   ├── auth.ts                # Auth.js configuration
│   │   ├── db.ts                  # Prisma client singleton
│   │   ├── role-check.ts          # Auth + role verification helpers
│   │   └── validations.ts         # Zod schemas
│   ├── middleware.ts              # Route protection
│   └── types/
│       └── next-auth.d.ts         # Auth.js type extensions
├── .env                           # Environment variables (NOT committed)
├── package.json
└── tsconfig.json
```

---

12. RBAC Implementation

RBAC = Role-Based Access Control. Different users see different things, can do different things.

Roles in Our App

Role	Can Do
USER	Create/edit/delete their OWN todos. View their own dashboard and charts.
ADMIN	Everything USER can do + view ALL users, ALL todos, site-wide analytics, delete any todo, change user roles.

Helper: Check Role in Server Components

```
// src/lib/role-check.ts

import { auth } from "@/lib/auth";
import { redirect } from "next/navigation";

// Use in any server component or server action to verify role
export async function requireAuth() {
  const session = await auth();
  if (!session?.user) redirect("/login");
  return session;
}

export async function requireAdmin() {
  const session = await requireAuth();
  if (session.user.role !== "ADMIN") redirect("/dashboard");
  return session;
}
```

Using Role Checks in Pages

```
// src/app/admin/page.tsx — Only ADMIN can see this
import { requireAdmin } from "@/lib/role-check";

export default async function AdminPage() {
  const session = await requireAdmin(); // Redirects non-admins automatically

  return <div>Welcome, Admin {session.user.name}</div>;
}
```

Conditional UI Based on Role

```
// In any server component
import { auth } from "@/lib/auth";

export default async function Navbar() {
  const session = await auth();

  return (
    <nav>
      <a href="/dashboard">Dashboard</a>
      {session?.user?.role === "ADMIN" && (
        <a href="/admin">Admin Panel</a>
      )}
    </nav>
  );
}
```

Making the First Admin

After registering your first user, manually promote them to admin:

```
npx prisma studio
```

1. Find your user in the User table
2. Change `role` from `USER` to `ADMIN`
3. Save

Or via a quick script:

```
// scripts/make-admin.ts — Run with: npx tsx scripts/make-admin.ts
import { PrismaClient } from "@prisma/client";
const prisma = new PrismaClient();

async function main() {
  await prisma.user.update({
    where: { email: "your@email.com" },
    data: { role: "ADMIN" },
  });
  console.log("Done — user is now ADMIN");
}

main();
```

---

13. Todo CRUD with Server Actions

Server Actions are functions that run on the server but can be called directly from client components. No API routes needed.

```
// src/actions/todo.ts

"use server";

import { revalidatePath } from "next/cache";
import { db } from "@/lib/db";
import { auth } from "@/lib/auth";
import { createTodoSchema, updateTodoSchema } from "@/lib/validations";

// ─── CREATE TODO ──────────────────────────────

export async function createTodo(formData: FormData) {
  const session = await auth();
  if (!session?.user?.id) return { error: "Not authenticated" };

  const rawData = {
    title: formData.get("title") as string,
    description: formData.get("description") as string,
    priority: formData.get("priority") as string,
  };

  // Validate input with Zod
  const validated = createTodoSchema.safeParse(rawData);
  if (!validated.success) {
    return { error: validated.error.errors[0].message };
  }

  await db.todo.create({
    data: {
      ...validated.data,
      userId: session.user.id,
    },
  });

  // Tell Next.js to refresh the dashboard page data
  revalidatePath("/dashboard");
  return { success: true };
}

// ─── TOGGLE TODO COMPLETE ─────────────────────

export async function toggleTodo(todoId: string) {
  const session = await auth();
  if (!session?.user?.id) return { error: "Not authenticated" };

  // Find the todo and verify ownership
  const todo = await db.todo.findUnique({ where: { id: todoId } });

  if (!todo) return { error: "Todo not found" };

  // Allow if owner OR admin
  if (todo.userId !== session.user.id && session.user.role !== "ADMIN") {
    return { error: "Not authorized" };
  }

  await db.todo.update({
    where: { id: todoId },
    data: { completed: !todo.completed },
  });

  revalidatePath("/dashboard");
  revalidatePath("/admin");
  return { success: true };
}
```

```
// ─── UPDATE TODO ──────────────────────────────

export async function updateTodo(formData: FormData) {
  const session = await auth();
  if (!session?.user?.id) return { error: "Not authenticated" };

  const rawData = {
    id: formData.get("id") as string,
    title: formData.get("title") as string,
    description: formData.get("description") as string,
    priority: formData.get("priority") as string,
  };

  const validated = updateTodoSchema.safeParse(rawData);
  if (!validated.success) {
    return { error: validated.error.errors[0].message };
  }

  // Verify ownership
  const existing = await db.todo.findUnique({ where: { id: validated.data.id } });
  if (!existing) return { error: "Todo not found" };
  if (existing.userId !== session.user.id && session.user.role !== "ADMIN") {
    return { error: "Not authorized" };
  }

  await db.todo.update({
    where: { id: validated.data.id },
    data: {
      title: validated.data.title,
      description: validated.data.description,
      priority: validated.data.priority as any,
    },
  });

  revalidatePath("/dashboard");
  return { success: true };
}

// ─── DELETE TODO ──────────────────────────────

export async function deleteTodo(todoId: string) {
  const session = await auth();
  if (!session?.user?.id) return { error: "Not authenticated" };

  const todo = await db.todo.findUnique({ where: { id: todoId } });
  if (!todo) return { error: "Todo not found" };

  // Only owner or admin can delete
  if (todo.userId !== session.user.id && session.user.role !== "ADMIN") {
    return { error: "Not authorized" };
  }

  await db.todo.delete({ where: { id: todoId } });

  revalidatePath("/dashboard");
  revalidatePath("/admin");
  return { success: true };
}

// ─── GET TODOS (for current user) ─────────────

export async function getUserTodos() {
  const session = await auth();
  if (!session?.user?.id) return [];

  return db.todo.findMany({
```

```
    where: { userId: session.user.id },
    orderBy: { createdAt: "desc" },
  });
}

// ─── GET ALL TODOS (admin only) ────────────────────

export async function getAllTodos() {
  const session = await auth();
  if (!session?.user?.id || session.user.role !== "ADMIN") return [];

  return db.todo.findMany({
    include: { user: { select: { name: true, email: true } } },
    orderBy: { createdAt: "desc" },
  });
}

// ─── GET STATS (for charts) ────────────────────────

export async function getUserStats() {
  const session = await auth();
  if (!session?.user?.id) return null;

  const [total, completed, byPriority, last7Days] = await Promise.all([
    db.todo.count({ where: { userId: session.user.id } }),
    db.todo.count({ where: { userId: session.user.id, completed: true } }),
    db.todo.groupBy({
      by: ["priority"],
      _count: { id: true },
      where: { userId: session.user.id },
    }),
    // Todos created in the last 7 days, grouped by day
    db.$queryRaw`
      SELECT DATE("createdAt") as date, COUNT(*)::int as count
      FROM "Todo"
      WHERE "userId" = ${session.user.id}
        AND "createdAt" >= NOW() - INTERVAL '7 days'
      GROUP BY DATE("createdAt")
      ORDER BY date
    ` as Promise<{ date: Date; count: number }[]>,
  ]);

  return { total, completed, pending: total - completed, byPriority, last7Days };
}

export async function getAdminStats() {
  const session = await auth();
  if (!session?.user?.id || session.user.role !== "ADMIN") return null;

  const [totalUsers, totalTodos, completedTodos, todosByUser] = await Promise.all([
    db.user.count(),
    db.todo.count(),
    db.todo.count({ where: { completed: true } }),
    db.todo.groupBy({
      by: ["userId"],
      _count: { id: true },
    }),
  ]);

  // Get user names for the chart
  const usersWithCounts = await Promise.all(
    todosByUser.map(async (item) => {
      const user = await db.user.findUnique({
        where: { id: item.userId },
        select: { name: true },
      });
```

```
      return { name: user?.name || "Unknown", count: item._count.id };
    })
  );

  return { totalUsers, totalTodos, completedTodos, pendingTodos: totalTodos - completedTodos, usersWithCounts };
}
```

---

14. Building the Pages

Root Layout

```
// src/app/layout.tsx

import type { Metadata } from "next";
import { Inter } from "next/font/google";
import "./globals.css";
import { Toaster } from "@/components/ui/sonner";
import { SessionProvider } from "next-auth/react";

const inter = Inter({ subsets: ["latin"] });

export const metadata: Metadata = {
  title: "Todo App",
  description: "Todo app with RBAC and analytics",
};

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en">
      <body className={inter.className}>
        <SessionProvider>
          {children}
        </SessionProvider>
        <Toaster />
      </body>
    </html>
  );
}
```

Landing Page

```
// src/app/page.tsx

import Link from "next/link";
import { Button } from "@/components/ui/button";
import { auth } from "@/lib/auth";
import { redirect } from "next/navigation";

export default async function Home() {
  const session = await auth();
  if (session) redirect("/dashboard");

  return (
    <main className="flex min-h-screen flex-col items-center justify-center gap-6 p-4">
      <h1 className="text-4xl font-bold">Todo App</h1>
      <p className="text-muted-foreground text-lg">
        Manage your tasks. Track your progress.
      </p>
      <div className="flex gap-4">
        <Link href="/login">
          <Button>Login</Button>
        </Link>
        <Link href="/register">
          <Button variant="outline">Register</Button>
        </Link>
      </div>
    </main>
  );
}
```

Login Page

```
// src/app/login/page.tsx

"use client";

import { useState } from "react";
import { useRouter } from "next/navigation";
import Link from "next/link";
import { signIn } from "next-auth/react";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from "@/components/ui/card";

export default function LoginPage() {
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  const router = useRouter();

  async function handleSubmit(e: React.FormEvent<HTMLFormElement>) {
    e.preventDefault();
    setLoading(true);
    setError("");

    const formData = new FormData(e.currentTarget);

    const result = await signIn("credentials", {
      email: formData.get("email"),
      password: formData.get("password"),
      redirect: false,
    });

    if (result?.error) {
      setError("Invalid email or password");
      setLoading(false);
    } else {
      router.push("/dashboard");
    }
  }

  return (
    <div className="flex min-h-screen items-center justify-center p-4">
      <Card className="w-full max-w-md">
        <CardHeader>
          <CardTitle>Login</CardTitle>
          <CardDescription>Enter your credentials to continue</CardDescription>
        </CardHeader>
        <CardContent>
          <form onSubmit={handleSubmit} className="space-y-4">
            {error && (
              <p className="text-sm text-red-500 bg-red-50 p-2 rounded">{error}</p>
            )}
            <div className="space-y-2">
              <Label htmlFor="email">Email</Label>
              <Input id="email" name="email" type="email" required />
            </div>
            <div className="space-y-2">
              <Label htmlFor="password">Password</Label>
              <Input id="password" name="password" type="password" required />
            </div>
            <Button type="submit" className="w-full" disabled={loading}>
              {loading ? "Logging in..." : "Login"}
            </Button>
          </form>

          <div className="mt-4 space-y-2">
```

```
            <div className="relative">
              <div className="absolute inset-0 flex items-center">
                <span className="w-full border-t" />
              </div>
              <div className="relative flex justify-center text-xs uppercase">
                <span className="bg-background px-2 text-muted-foreground">Or</span>
              </div>
            </div>
            <Button
              variant="outline"
              className="w-full"
              onClick={() => signIn("github", { callbackUrl: "/dashboard" })}
            >
              Continue with GitHub
            </Button>
            <Button
              variant="outline"
              className="w-full"
              onClick={() => signIn("google", { callbackUrl: "/dashboard" })}
            >
              Continue with Google
            </Button>
          </div>

          <p className="mt-4 text-center text-sm text-muted-foreground">
            No account?{" "}
            <Link href="/register" className="text-primary underline">
              Register
            </Link>
          </p>
        </CardContent>
      </Card>
    </div>
  );
}
```

Register Page

```
// src/app/register/page.tsx

"use client";

import { useState } from "react";
import Link from "next/link";
import { registerUser } from "@/actions/auth";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from "@/components/ui/card";

export default function RegisterPage() {
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);

  async function handleSubmit(e: React.FormEvent<HTMLFormElement>) {
    e.preventDefault();
    setLoading(true);
    setError("");

    const formData = new FormData(e.currentTarget);
    const result = await registerUser(formData);

    if (result?.error) {
      setError(result.error);
      setLoading(false);
    }
    // If successful, registerUser redirects to /dashboard automatically
  }

  return (
    <div className="flex min-h-screen items-center justify-center p-4">
      <Card className="w-full max-w-md">
        <CardHeader>
          <CardTitle>Create Account</CardTitle>
          <CardDescription>Sign up to start managing your todos</CardDescription>
        </CardHeader>
        <CardContent>
          <form onSubmit={handleSubmit} className="space-y-4">
            {error && (
              <p className="text-sm text-red-500 bg-red-50 p-2 rounded">{error}</p>
            )}
            <div className="space-y-2">
              <Label htmlFor="name">Name</Label>
              <Input id="name" name="name" required />
            </div>
            <div className="space-y-2">
              <Label htmlFor="email">Email</Label>
              <Input id="email" name="email" type="email" required />
            </div>
            <div className="space-y-2">
              <Label htmlFor="password">Password</Label>
              <Input id="password" name="password" type="password" required minLength={6} />
            </div>
            <Button type="submit" className="w-full" disabled={loading}>
              {loading ? "Creating account..." : "Register"}
            </Button>
          </form>
          <p className="mt-4 text-center text-sm text-muted-foreground">
            Already have an account?{" "}
            <Link href="/login" className="text-primary underline">
              Login
            </Link>
          </p>
```

```
        </CardContent>
      </Card>
    </div>
  );
}
```

Dashboard Page (User)

```
// src/app/dashboard/page.tsx

import { requireAuth } from "@/lib/role-check";
import { getUserTodos, getUserStats } from "@/actions/todo";
import { TodoList } from "@/components/todo-list";
import { TodoForm } from "@/components/todo-form";
import { StatsCards } from "@/components/stats-cards";
import { UserChart } from "@/components/analytics-chart";

export default async function DashboardPage() {
  const session = await requireAuth();
  const [todos, stats] = await Promise.all([getUserTodos(), getUserStats()]);

  return (
    <div className="container mx-auto p-6 space-y-6">
      <div className="flex items-center justify-between">
        <div>
          <h1 className="text-3xl font-bold">Dashboard</h1>
          <p className="text-muted-foreground">Welcome, {session.user.name}</p>
        </div>
        <TodoForm />
      </div>

      {/* Stats Cards */}
      {stats && <StatsCards stats={stats} />}

      {/* Chart */}
      {stats && <UserChart stats={stats} />}

      {/* Todo List */}
      <TodoList todos={todos} />
    </div>
  );
}
```

Dashboard Layout (with Sidebar + Header)

```
// src/app/dashboard/layout.tsx

import { auth } from "@/lib/auth";
import { redirect } from "next/navigation";
import { UserButton } from "@/components/user-button";
import Link from "next/link";

export default async function DashboardLayout({ children }: { children: React.ReactNode }) {
  const session = await auth();
  if (!session) redirect("/login");

  return (
    <div className="min-h-screen bg-gray-50">
      {/* Top Nav */}
      <header className="sticky top-0 z-50 border-b bg-white">
        <div className="container mx-auto flex h-14 items-center justify-between px-6">
          <div className="flex items-center gap-6">
            <Link href="/dashboard" className="font-bold text-lg">
              TodoApp
            </Link>
            <nav className="flex gap-4 text-sm">
              <Link href="/dashboard" className="text-muted-foreground hover:text-foreground">
                My Todos
              </Link>
              {session.user.role === "ADMIN" && (
                <Link href="/admin" className="text-muted-foreground hover:text-foreground">
                  Admin Panel
                </Link>
              )}
            </nav>
          </div>
          <UserButton user={session.user} />
        </div>
      </header>

      {/* Page Content */}
      <main>{children}</main>
    </div>
  );
}
```

Admin Dashboard Page

```
// src/app/admin/page.tsx

import { requireAdmin } from "@/lib/role-check";
import { getAllTodos, getAdminStats } from "@/actions/todo";
import { AdminChart } from "@/components/analytics-chart";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import {
  Table,
  TableBody,
  TableCell,
  TableHead,
  TableHeader,
  TableRow,
} from "@/components/ui/table";

export default async function AdminPage() {
  const session = await requireAdmin();
  const [todos, stats] = await Promise.all([getAllTodos(), getAdminStats()]);

  return (
    <div className="container mx-auto p-6 space-y-6">
      <h1 className="text-3xl font-bold">Admin Dashboard</h1>

      {/* Stats Overview */}
      {stats && (
        <div className="grid grid-cols-1 md:grid-cols-4 gap-4">
          <Card>
            <CardHeader className="pb-2">
              <CardTitle className="text-sm text-muted-foreground">Total Users</CardTitle>
            </CardHeader>
            <CardContent>
              <p className="text-3xl font-bold">{stats.totalUsers}</p>
            </CardContent>
          </Card>
          <Card>
            <CardHeader className="pb-2">
              <CardTitle className="text-sm text-muted-foreground">Total Todos</CardTitle>
            </CardHeader>
            <CardContent>
              <p className="text-3xl font-bold">{stats.totalTodos}</p>
            </CardContent>
          </Card>
          <Card>
            <CardHeader className="pb-2">
              <CardTitle className="text-sm text-muted-foreground">Completed</CardTitle>
            </CardHeader>
            <CardContent>
              <p className="text-3xl font-bold text-green-600">{stats.completedTodos}</p>
            </CardContent>
          </Card>
          <Card>
            <CardHeader className="pb-2">
              <CardTitle className="text-sm text-muted-foreground">Pending</CardTitle>
            </CardHeader>
            <CardContent>
              <p className="text-3xl font-bold text-orange-500">{stats.pendingTodos}</p>
            </CardContent>
          </Card>
        </div>
      )}

      {/* Admin Charts */}
      {stats && <AdminChart stats={stats} />}
```

```
      {/* All Todos Table */}
      <Card>
        <CardHeader>
          <CardTitle>All Todos</CardTitle>
        </CardHeader>
        <CardContent>
          <Table>
            <TableHeader>
              <TableRow>
                <TableHead>Title</TableHead>
                <TableHead>User</TableHead>
                <TableHead>Priority</TableHead>
                <TableHead>Status</TableHead>
                <TableHead>Created</TableHead>
              </TableRow>
            </TableHeader>
            <TableBody>
              {todos.map((todo: any) => (
                <TableRow key={todo.id}>
                  <TableCell className="font-medium">{todo.title}</TableCell>
                  <TableCell>{todo.user?.name || todo.user?.email}</TableCell>
                  <TableCell>
                    <Badge
                      variant={
                        todo.priority === "HIGH"
                          ? "destructive"
                          : todo.priority === "MEDIUM"
                          ? "default"
                          : "secondary"
                      }
                    >
                      {todo.priority}
                    </Badge>
                  </TableCell>
                  <TableCell>
                    <Badge variant={todo.completed ? "default" : "outline"}>
                      {todo.completed ? "Done" : "Pending"}
                    </Badge>
                  </TableCell>
                  <TableCell className="text-muted-foreground">
                    {new Date(todo.createdAt).toLocaleDateString()}
                  </TableCell>
                </TableRow>
              ))}
            </TableBody>
          </Table>
        </CardContent>
      </Card>
    </div>
  );
}
```

Admin Layout

```
// src/app/admin/layout.tsx

import { auth } from "@/lib/auth";
import { redirect } from "next/navigation";
import { UserButton } from "@/components/user-button";
import Link from "next/link";

export default async function AdminLayout({ children }: { children: React.ReactNode }) {
  const session = await auth();
  if (!session) redirect("/login");
  if (session.user.role !== "ADMIN") redirect("/dashboard");

  return (
    <div className="min-h-screen bg-gray-50">
      <header className="sticky top-0 z-50 border-b bg-white">
        <div className="container mx-auto flex h-14 items-center justify-between px-6">
          <div className="flex items-center gap-6">
            <Link href="/dashboard" className="font-bold text-lg">
              TodoApp
            </Link>
            <nav className="flex gap-4 text-sm">
              <Link href="/dashboard" className="text-muted-foreground hover:text-foreground">
                My Todos
              </Link>
              <Link href="/admin" className="text-muted-foreground hover:text-foreground font-medium">
                Admin Panel
              </Link>
            </nav>
          </div>
          <UserButton user={session.user} />
        </div>
      </header>
      <main>{children}</main>
    </div>
  );
}
```

---

15. Chart.js Analytics Dashboard

Install + Register Chart.js

```
// src/components/analytics-chart.tsx

"use client";

import {
  Chart as ChartJS,
  CategoryScale,
  LinearScale,
  BarElement,
  ArcElement,
  Title,
  Tooltip,
  Legend,
} from "chart.js";
import { Bar, Pie } from "react-chartjs-2";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";

// Register Chart.js components (required — Chart.js is tree-shakeable)
ChartJS.register(CategoryScale, LinearScale, BarElement, ArcElement, Title, Tooltip, Legend);

// ─── USER DASHBOARD CHARTS ──────────────────────

interface UserStats {
  total: number;
  completed: number;
  pending: number;
  byPriority: { priority: string; _count: { id: number } }[];
  last7Days: { date: Date; count: number }[];
}

export function UserChart({ stats }: { stats: UserStats }) {
  // Pie chart: Completed vs Pending
  const pieData = {
    labels: ["Completed", "Pending"],
    datasets: [
      {
        data: [stats.completed, stats.pending],
        backgroundColor: ["#22c55e", "#f97316"],
        borderWidth: 0,
      },
    ],
  };

  // Bar chart: Todos created per day (last 7 days)
  const barData = {
    labels: stats.last7Days.map((d) =>
      new Date(d.date).toLocaleDateString("en-US", { weekday: "short" })
    ),
    datasets: [
      {
        label: "Todos Created",
        data: stats.last7Days.map((d) => d.count),
        backgroundColor: "#3b82f6",
        borderRadius: 6,
      },
    ],
  };

  const barOptions = {
    responsive: true,
    plugins: { legend: { display: false } },
    scales: {
      y: { beginAtZero: true, ticks: { stepSize: 1 } },
    },
  };
```

```
  return (
    <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
      <Card>
        <CardHeader>
          <CardTitle className="text-lg">Completion Rate</CardTitle>
        </CardHeader>
        <CardContent className="flex justify-center">
          <div className="w-64 h-64">
            <Pie data={pieData} />
          </div>
        </CardContent>
      </Card>

      <Card>
        <CardHeader>
          <CardTitle className="text-lg">Todos This Week</CardTitle>
        </CardHeader>
        <CardContent>
          <Bar data={barData} options={barOptions} />
        </CardContent>
      </Card>
    </div>
  );
}

// ─── ADMIN DASHBOARD CHARTS ──────────────────────

interface AdminStats {
  totalUsers: number;
  totalTodos: number;
  completedTodos: number;
  pendingTodos: number;
  usersWithCounts: { name: string; count: number }[];
}

export function AdminChart({ stats }: { stats: AdminStats }) {
  // Bar chart: Todos per user
  const barData = {
    labels: stats.usersWithCounts.map((u) => u.name),
    datasets: [
      {
        label: "Todos",
        data: stats.usersWithCounts.map((u) => u.count),
        backgroundColor: "#8b5cf6",
        borderRadius: 6,
      },
    ],
  };

  const barOptions = {
    responsive: true,
    plugins: { legend: { display: false } },
    scales: {
      y: { beginAtZero: true, ticks: { stepSize: 1 } },
    },
  };

  // Pie chart: Overall completion
  const pieData = {
    labels: ["Completed", "Pending"],
    datasets: [
      {
        data: [stats.completedTodos, stats.pendingTodos],
        backgroundColor: ["#22c55e", "#f97316"],
        borderWidth: 0,
```

```
      },
    ],
  };

  return (
    <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
      <Card>
        <CardHeader>
          <CardTitle className="text-lg">Todos Per User</CardTitle>
        </CardHeader>
        <CardContent>
          <Bar data={barData} options={barOptions} />
        </CardContent>
      </Card>

      <Card>
        <CardHeader>
          <CardTitle className="text-lg">Overall Completion</CardTitle>
        </CardHeader>
        <CardContent className="flex justify-center">
          <div className="w-64 h-64">
            <Pie data={pieData} />
          </div>
        </CardContent>
      </Card>
    </div>
  );
}
```

Stats Cards Component

```
// src/components/stats-cards.tsx

import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";

interface StatsCardsProps {
  stats: {
    total: number;
    completed: number;
    pending: number;
  };
}

export function StatsCards({ stats }: StatsCardsProps) {
  return (
    <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
      <Card>
        <CardHeader className="pb-2">
          <CardTitle className="text-sm text-muted-foreground">Total Todos</CardTitle>
        </CardHeader>
        <CardContent>
          <p className="text-3xl font-bold">{stats.total}</p>
        </CardContent>
      </Card>
      <Card>
        <CardHeader className="pb-2">
          <CardTitle className="text-sm text-muted-foreground">Completed</CardTitle>
        </CardHeader>
        <CardContent>
          <p className="text-3xl font-bold text-green-600">{stats.completed}</p>
        </CardContent>
      </Card>
      <Card>
        <CardHeader className="pb-2">
          <CardTitle className="text-sm text-muted-foreground">Pending</CardTitle>
        </CardHeader>
        <CardContent>
          <p className="text-3xl font-bold text-orange-500">{stats.pending}</p>
        </CardContent>
      </Card>
    </div>
  );
}
```

Todo List Component

```
// src/components/todo-list.tsx

"use client";

import { toggleTodo, deleteTodo } from "@/actions/todo";
import { Button } from "@/components/ui/button";
import { Card, CardContent } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";

interface Todo {
  id: string;
  title: string;
  description: string | null;
  completed: boolean;
  priority: "LOW" | "MEDIUM" | "HIGH";
  createdAt: Date;
}

export function TodoList({ todos }: { todos: Todo[] }) {
  if (todos.length === 0) {
    return (
      <div className="text-center py-12 text-muted-foreground">
        No todos yet. Create your first one!
      </div>
    );
  }

  return (
    <div className="space-y-3">
      {todos.map((todo) => (
        <Card key={todo.id} className={todo.completed ? "opacity-60" : ""}>
          <CardContent className="flex items-center justify-between p-4">
            <div className="flex items-center gap-4">
              <input
                type="checkbox"
                checked={todo.completed}
                onChange={() => toggleTodo(todo.id)}
                className="h-5 w-5 rounded cursor-pointer"
              />
              <div>
                <p className={`font-medium ${todo.completed ? "line-through" : ""}`}>
                  {todo.title}
                </p>
                {todo.description && (
                  <p className="text-sm text-muted-foreground">{todo.description}</p>
                )}
              </div>
            </div>
            <div className="flex items-center gap-2">
              <Badge
                variant={
                  todo.priority === "HIGH"
                    ? "destructive"
                    : todo.priority === "MEDIUM"
                    ? "default"
                    : "secondary"
                }
              >
                {todo.priority}
              </Badge>
              <Button
                variant="ghost"
                size="sm"
                onClick={() => deleteTodo(todo.id)}
                className="text-red-500 hover:text-red-700"
```

```
              >
                Delete
              </Button>
            </div>
          </CardContent>
        </Card>
      ))}
    </div>
  );
}
```

Todo Form Component (Create Dialog)

```
// src/components/todo-form.tsx

"use client";

import { useState } from "react";
import { createTodo } from "@/actions/todo";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Textarea } from "@/components/ui/textarea";
import {
  Dialog,
  DialogContent,
  DialogHeader,
  DialogTitle,
  DialogTrigger,
} from "@/components/ui/dialog";
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from "@/components/ui/select";

export function TodoForm() {
  const [open, setOpen] = useState(false);
  const [error, setError] = useState("");

  async function handleSubmit(e: React.FormEvent<HTMLFormElement>) {
    e.preventDefault();
    setError("");

    const formData = new FormData(e.currentTarget);
    const result = await createTodo(formData);

    if (result?.error) {
      setError(result.error);
    } else {
      setOpen(false);
    }
  }

  return (
    <Dialog open={open} onOpenChange={setOpen}>
      <DialogTrigger asChild>
        <Button>+ New Todo</Button>
      </DialogTrigger>
      <DialogContent>
        <DialogHeader>
          <DialogTitle>Create Todo</DialogTitle>
        </DialogHeader>
        <form onSubmit={handleSubmit} className="space-y-4">
          {error && (
            <p className="text-sm text-red-500 bg-red-50 p-2 rounded">{error}</p>
          )}
          <div className="space-y-2">
            <Label htmlFor="title">Title</Label>
            <Input id="title" name="title" required />
          </div>
          <div className="space-y-2">
            <Label htmlFor="description">Description (optional)</Label>
            <Textarea id="description" name="description" />
          </div>
          <div className="space-y-2">
```

```
            <Label htmlFor="priority">Priority</Label>
            <Select name="priority" defaultValue="MEDIUM">
              <SelectTrigger>
                <SelectValue />
              </SelectTrigger>
              <SelectContent>
                <SelectItem value="LOW">Low</SelectItem>
                <SelectItem value="MEDIUM">Medium</SelectItem>
                <SelectItem value="HIGH">High</SelectItem>
              </SelectContent>
            </Select>
          </div>
          <Button type="submit" className="w-full">
            Create Todo
          </Button>
        </form>
      </DialogContent>
    </Dialog>
  );
}
```

User Button Component

```
// src/components/user-button.tsx

"use client";

import { signOut } from "next-auth/react";
import { Button } from "@/components/ui/button";
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuTrigger,
} from "@/components/ui/dropdown-menu";
import { Avatar, AvatarFallback, AvatarImage } from "@/components/ui/avatar";

interface UserButtonProps {
  user: {
    name?: string | null;
    email?: string | null;
    image?: string | null;
    role?: string;
  };
}

export function UserButton({ user }: UserButtonProps) {
  const initials = user.name
    ?.split(" ")
    .map((n) => n[0])
    .join("")
    .toUpperCase() || "U";

  return (
    <DropdownMenu>
      <DropdownMenuTrigger asChild>
        <Button variant="ghost" className="relative h-8 w-8 rounded-full">
          <Avatar className="h-8 w-8">
            <AvatarImage src={user.image || ""} />
            <AvatarFallback>{initials}</AvatarFallback>
          </Avatar>
        </Button>
      </DropdownMenuTrigger>
      <DropdownMenuContent align="end">
        <div className="flex items-center gap-2 p-2">
          <div className="flex flex-col space-y-1">
            <p className="text-sm font-medium">{user.name}</p>
            <p className="text-xs text-muted-foreground">{user.email}</p>
          </div>
        </div>
        <DropdownMenuItem onClick={() => signOut({ callbackUrl: "/" })}>
          Log out
        </DropdownMenuItem>
      </DropdownMenuContent>
    </DropdownMenu>
  );
}
```

16. Tailwind Patterns

Common Layouts

```
// Centered content
<div className="flex min-h-screen items-center justify-center">
  {/* content */}
</div>

// Container with max width
<div className="container mx-auto px-4 py-6">
  {/* content */}
</div>

// Responsive grid: 1 col mobile, 2 cols tablet, 3 cols desktop
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
  {/* cards */}
</div>

// Sticky header
<header className="sticky top-0 z-50 border-b bg-white/80 backdrop-blur">
  {/* nav */}
</header>

// Flex row with space between
<div className="flex items-center justify-between">
  <h1>Title</h1>
  <Button>Action</Button>
</div>

// Card-like container
<div className="rounded-lg border bg-card p-6 shadow-sm">
  {/* content */}
</div>

// Responsive sidebar layout
<div className="flex">
  <aside className="hidden md:block w-64 border-r p-4">Sidebar</aside>
  <main className="flex-1 p-6">Content</main>
</div>
```

Spacing System

```
p-1 = 4px    p-2 = 8px    p-3 = 12px   p-4 = 16px
p-5 = 20px   p-6 = 24px   p-8 = 32px   p-10 = 40px
p-12 = 48px  p-16 = 64px  p-20 = 80px  p-24 = 96px

Same values for: m (margin), gap, space-x, space-y
```

Text Sizes

```
text-xs = 12px    text-sm = 14px    text-base = 16px
text-lg = 18px    text-xl = 20px    text-2xl = 24px
text-3xl = 30px   text-4xl = 36px
```

Dark Mode (optional)

```
// In tailwind.config.ts — add dark mode
module.exports = {
  darkMode: "class",
  // ...
};

// Toggle button component
"use client";
import { useTheme } from "next-themes";
import { Button } from "@/components/ui/button";

export function ThemeToggle() {
  const { theme, setTheme } = useTheme();
  return (
    <Button variant="ghost" onClick={() => setTheme(theme === "dark" ? "light" : "dark")}>
      {theme === "dark" ? "☀" : "□"}
    </Button>
  );
}

// Usage in classes: dark:bg-gray-900 dark:text-white
```

---

17. Vercel Deployment

Step 1: Add Prisma Postinstall Script

Vercel needs to generate the Prisma client during build. Add this to `package.json`:

```
{
  "scripts": {
    "postinstall": "prisma generate",
    "build": "next build",
    "dev": "next dev --turbopack",
    "start": "next start"
  }
}
```

Why? Vercel runs `npm install` (which triggers `postinstall`) then `npm run build`. Without `postinstall`, Prisma Client won't exist and the build fails.

Step 2: Deploy via Vercel CLI

```
# Install Vercel CLI
npm i -g vercel

# Login
vercel login

# Deploy (first time — links to your Vercel account)
vercel

# Deploy to production
vercel --prod
```

Step 3: Set Environment Variables on Vercel

Option A: CLI

```
vercel env add DATABASE_URL
vercel env add DIRECT_URL
vercel env add AUTH_SECRET
vercel env add AUTH_GOOGLE_ID
vercel env add AUTH_GOOGLE_SECRET
vercel env add AUTH_GITHUB_ID
vercel env add AUTH_GITHUB_SECRET
```

Option B: Vercel Dashboard

1. Go to https://vercel.com (https://vercel.com) → Your project → Settings → Environment Variables
2. Add each variable from your `.env` file
3. Make sure they're enabled for Production, Preview, and Development

Step 4: Connect GitHub for Auto-Deploy

1. Vercel Dashboard → Your Project → Settings → Git
2. Connect your GitHub repo
3. Now every `git push` to `main` auto-deploys to production
4. Every PR gets a preview deployment URL

Step 5: Add Auth Callback URLs

After deploying, update your OAuth providers with the production URLs:

Google Cloud Console:

```
Authorized redirect URIs:
https://your-app.vercel.app/api/auth/callback/google
http://localhost:3000/api/auth/callback/google
```

GitHub Developer Settings:

```
Authorization callback URL:
https://your-app.vercel.app/api/auth/callback/github
```

Also add to `.env` / Vercel env vars:

```
AUTH_URL=https://your-app.vercel.app
```

---

18. Quick Reference Command Table

One-Time Setup Commands

Command	What It Does
`npx create-next-app@latest my-app`	Create new Next.js project
`cd my-app`	Enter project folder
`npm install next-auth@beta @auth/prisma-adapter`	Auth library + Prisma adapter
`npm install @prisma/client`	Prisma ORM client
`npm install prisma --save-dev`	Prisma CLI (dev tool)
`npm install zod`	Validation library
`npm install bcryptjs`	Password hashing
`npm install @types/bcryptjs -D`	TypeScript types for bcrypt
`npm install react-chartjs-2 chart.js`	Charts
`npx shadcn@latest init`	Initialize shadcn/ui
`npx shadcn@latest add button card input ...`	Add UI components
`npx prisma init`	Initialize Prisma (creates schema.prisma)
`npx auth secret`	Generate AUTH_SECRET for .env
`gh repo create my-app --public --source=. --remote=origin --push`	Create GitHub repo + push

Everyday Dev Commands

Command	What It Does

Command	What It Does
`npm run dev`	Start dev server at localhost:3000
`npx prisma studio`	Open visual DB browser
`npx prisma migrate dev --name describe_change`	Create + apply migration
`npx prisma db push`	Quick-push schema (no migration file)
`npx prisma generate`	Regenerate Prisma Client types
`npx prisma migrate reset`	WIPE DB + re-run all migrations
`npx prisma format`	Format schema file
`npx shadcn@latest add [component]`	Add a new shadcn component

Deployment Commands

Command	What It Does
`vercel`	Deploy to preview
`vercel --prod`	Deploy to production
`vercel env add VAR_NAME`	Add environment variable
`vercel logs`	View deployment logs

Git Commands

Command	What It Does
`git add . && git commit -m "feat: message" && git push`	Stage + commit + push
`git checkout -b feature/name`	Create new branch
`git merge feature/name`	Merge branch into current
`git stash`	Temporarily save uncommitted changes
`git stash pop`	Restore stashed changes
`git log --oneline -10`	View last 10 commits

---

19. Common Errors + Fixes

Error: PrismaClient is not configured for Edge Runtime

```
// FIX: Make sure you're NOT importing db.ts in middleware.ts
// Middleware runs on Edge Runtime, Prisma needs Node.js runtime
// Use Auth.js's built-in middleware helper instead of direct DB calls
```

Error: NEXTAUTH_URL is not set

```
# Add to .env:
AUTH_URL=http://localhost:3000

# On Vercel, add:
AUTH_URL=https://your-app.vercel.app
```

Error: Cannot find module '@prisma/client'

```
npx prisma generate
```

Error: relation "User" does not exist

```
# Your migrations haven't been applied
npx prisma migrate dev
```

Error: Invalid 'prisma.user.create()' invocation — Unique constraint failed on the fields: ('email')

```
A user with that email already exists. Check your register function for duplicate email handling.
```

Module not found: Can't resolve '@/components/ui/button'

```
# You haven't added the component yet
npx shadcn@latest add button
```

Hot reload not working

```
# Stop the dev server, delete .next folder, restart
rm -rf .next
npm run dev
```

Prisma schema out of sync

```
# Nuclear option — reset everything
npx prisma migrate reset
npx prisma migrate dev --name fresh_start
```

---

20. Hackathon Checklist

Use this during the hackathon to make sure you're not missing anything.

Setup Phase (30 min)

- [ ] `npx create-next-app@latest` with TypeScript + Tailwind + App Router
- [ ] Create GitHub repo + first push
- [ ] Create Supabase project + get connection strings
- [ ] Create `.env` with all variables
- [ ] `npm install` all dependencies (auth, prisma, zod, charts, bcrypt)
- [ ] `npx prisma init` + write schema
- [ ] `npx prisma migrate dev --name init`
- [ ] `npx shadcn@latest init` + add components
- [ ] Set up `src/lib/db.ts` (Prisma singleton)
- [ ] Set up `src/lib/auth.ts` (Auth.js config)
- [ ] Set up `src/middleware.ts` (route protection)
- [ ] Set up `src/types/next-auth.d.ts` (type extensions)
- [ ] Create `src/app/api/auth/[...nextauth]/route.ts`
- [ ] Git commit: `setup: initial project setup`

Core Features (2-3 hours)

- [ ] Register page + server action
- [ ] Login page + OAuth buttons
- [ ] Dashboard layout (header + sidebar)
- [ ] CRUD server actions with Zod validation
- [ ] Main feature pages
- [ ] Role-based access (middleware + UI)
- [ ] Git commit after each feature

Polish (1 hour)

- [ ] Loading states (`loading.tsx` files)
- [ ] Error handling (`error.tsx` files)
- [ ] Mobile responsive check
- [ ] Charts/analytics dashboard
- [ ] Empty states (no data messages)

Deploy (15 min)

- [ ] Add `"postinstall": "prisma generate"` to package.json
- [ ] `vercel` → set environment variables
- [ ] `vercel --prod`
- [ ] Test production URL

- [ ] Update OAuth callback URLs with production domain
- [ ] Final git push

Demo Prep (15 min)

- [ ] Seed demo data via Prisma Studio
- [ ] Create admin account + regular account
- [ ] Test full flow: register → login → create → complete → admin view
- [ ] Have Prisma Studio open to show DB during demo

Seed Script (Populate Demo Data)

Create this file to quickly fill your app with demo data for presentations:

```
// prisma/seed.ts

import { PrismaClient } from "@prisma/client";
import bcrypt from "bcryptjs";

const prisma = new PrismaClient();

async function main() {
  // Clean existing data
  await prisma.todo.deleteMany();
  await prisma.user.deleteMany();

  const hashedPassword = await bcrypt.hash("password123", 10);

  // Create admin
  const admin = await prisma.user.create({
    data: {
      name: "Admin User",
      email: "admin@demo.com",
      password: hashedPassword,
      role: "ADMIN",
    },
  });

  // Create regular user
  const user = await prisma.user.create({
    data: {
      name: "John Doe",
      email: "john@demo.com",
      password: hashedPassword,
      role: "USER",
    },
  });

  // Create todos for both users
  const todosData = [
    { title: "Set up project repository", completed: true, priority: "HIGH" as const, userId: admin.id },
    { title: "Design database schema", completed: true, priority: "HIGH" as const, userId: admin.id },
    { title: "Implement authentication", completed: true, priority: "HIGH" as const, userId: admin.id },
    { title: "Build dashboard UI", completed: false, priority: "MEDIUM" as const, userId: admin.id },
    { title: "Add analytics charts", completed: false, priority: "LOW" as const, userId: admin.id },
    { title: "Write documentation", completed: false, priority: "LOW" as const, userId: admin.id },
    { title: "Buy groceries", completed: false, priority: "MEDIUM" as const, userId: user.id },
    { title: "Finish homework", completed: true, priority: "HIGH" as const, userId: user.id },
    { title: "Call dentist", completed: false, priority: "LOW" as const, userId: user.id },
    { title: "Prepare presentation", completed: false, priority: "HIGH" as const, userId: user.id },
    { title: "Read chapter 5", completed: true, priority: "MEDIUM" as const, userId: user.id },
    { title: "Update resume", completed: false, priority: "MEDIUM" as const, userId: user.id },
  ];

  for (const todo of todosData) {
    await prisma.todo.create({ data: todo });
  }

  console.log("Seeded: 2 users + 12 todos");
  console.log("Admin login: admin@demo.com / password123");
  console.log("User login:  john@demo.com / password123");
}

main()
  .catch(console.error)
  .finally(() => prisma.$disconnect());
```

Add to `package.json`:

```
{
  "prisma": {
    "seed": "npx tsx prisma/seed.ts"
  }
}
```

Run it:

```
# Install tsx (runs TypeScript directly)
npm install tsx --save-dev

# Seed the database
npx prisma db seed
```

Quick Copy-Paste: Full Setup in Order

Run these commands in sequence for a complete setup:

```
# 1. Create project
npx create-next-app@latest my-todo-app
cd my-todo-app

# 2. Install everything
npm install next-auth@beta @auth/prisma-adapter @prisma/client zod bcryptjs react-chartjs-2 chart.js
npm install prisma @types/bcryptjs tsx --save-dev

# 3. Init Prisma
npx prisma init

# 4. Init shadcn
npx shadcn@latest init

# 5. Add shadcn components
npx shadcn@latest add button card input label dialog table badge select textarea tabs separator avatar dropdown-menu sheet sonner

# 6. Generate auth secret
npx auth secret

# 7. (Edit .env with Supabase URLs)
# 8. (Write prisma/schema.prisma)

# 9. Run migration
npx prisma migrate dev --name init

# 10. (Create all the files from sections 7-15)

# 11. Start dev server
npm run dev

# 12. Git + GitHub
git add .
git commit -m "feat: complete todo app with RBAC"
gh repo create my-todo-app --public --source=. --remote=origin --push

# 13. Deploy
vercel --prod
```

That's it. Every command. Every file. Every "why." Go build something.